

ĐẠI HỌC BÁCH KHOA HÀ NỘI  
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



**BÁO CÁO BÀI TẬP LỚN**  
**NGUYÊN LÝ HỆ ĐIỀU HÀNH**  
**TỐI ƯU HÓA LẬP LỊCH TÁC VỤ**  
**TRONG TRUNG TÂM DỮ LIỆU**  
**DỰA TRÊN MÔ HÌNH HỌC MÁY**  
**SVR**

GVHD: Đỗ Tuấn Anh  
Sinh viên thực hiện: Hoàng Trọng Tú  
MSSV: 202416631

Hà Nội, Tháng 06 Năm 2026

# Mục lục

|                                                                  |           |
|------------------------------------------------------------------|-----------|
| <b>LỜI NÓI ĐẦU</b>                                               | <b>2</b>  |
| <b>1 GIỚI THIỆU</b>                                              | <b>3</b>  |
| 1.1 Bối cảnh nghiên cứu . . . . .                                | 3         |
| 1.2 Phương pháp đề xuất . . . . .                                | 3         |
| <b>2 KIẾN TRÚC HỆ THỐNG VÀ TIỀN XỬ LÝ DỮ LIỆU</b>                | <b>4</b>  |
| 2.1 Kiến trúc Phân tán Master-Agent . . . . .                    | 4         |
| 2.2 Pipeline Tiền xử lý dữ liệu với Pandas . . . . .             | 5         |
| 2.2.1 Cấu trúc chi tiết của các đặc trưng đầu vào . . . . .      | 5         |
| 2.2.2 Kỹ thuật mã hóa One-Hot Encoding phi tuyến . . . . .       | 5         |
| 2.2.3 Cấu trúc thuật toán tiền xử lý dữ liệu . . . . .           | 6         |
| <b>3 HUẤN LUYỆN MÔ HÌNH MÁY HỌC SVR</b>                          | <b>7</b>  |
| 3.1 Cơ sở toán học của Support Vector Regression (SVR) . . . . . | 7         |
| 3.2 Hiện thực hóa huấn luyện bằng Scikit-Learn . . . . .         | 7         |
| <b>4 KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ</b>                         | <b>8</b>  |
| 4.1 Chỉ số đánh giá độ chính xác của mô hình học máy . . . . .   | 8         |
| 4.2 Chỉ số đánh giá độ chính xác của AI . . . . .                | 9         |
| 4.3 Hiệu năng Lập lịch Hệ thống (System Metrics) . . . . .       | 9         |
| 4.3.1 Thời gian chờ trung bình (Average Wait Time) . . . . .     | 9         |
| 4.3.2 Sản lượng hệ thống (Throughput) . . . . .                  | 9         |
| <b>5 KẾT LUẬN</b>                                                | <b>11</b> |

# LỜI NÓI ĐẦU

Trong kỷ nguyên số hóa mạnh mẽ ngày nay, sự bùng nổ của dữ liệu lớn (Big Data), điện toán đám mây (Cloud Computing) và các mô hình trí tuệ nhân tạo (AI/Deep Learning) đã thúc đẩy các trung tâm dữ liệu (Datacenters) quy mô lớn phát triển với tốc độ chóng mặt. Trong môi trường này, hàng ngàn tác vụ (Jobs) với các đặc trưng tài nguyên (CPU, RAM) và tính chất công việc vô cùng đa dạng liên tục được gửi tới hệ thống mỗi giây. Làm thế nào để phân phối tài nguyên phần cứng một cách tối ưu, giảm thiểu thời gian chờ đợi của người dùng và nâng cao sản lượng xử lý của toàn hệ thống là một bài toán tối ưu hóa sống còn, quyết định hiệu năng vận hành và chi phí kinh tế của các nhà cung cấp dịch vụ.

Các thuật toán lập lịch truyền thống như Dominant Resource Fairness (DRF) dù giải quyết tốt bài toán chia sẻ tài nguyên công bằng, nhưng lại chưa thể tối ưu hóa tổng thời gian hoàn thành công việc. Trong khi đó, thuật toán Công việc ngắn nhất chạy trước — Shortest Job First (SJF) — đã được chứng minh về mặt toán học là giải pháp tối ưu nhất để đạt sản lượng tối đa (Throughput-optimal). Tuy nhiên, một rào cản mang tính kinh điển khiến SJF khó áp dụng trong các hệ thống thực tế là nó đòi hỏi thông tin tiên nghiệm (A priori) về thời gian chạy thực tế (Runtime) của các tác vụ ngay tại thời điểm chúng bước vào hàng đợi. Trong một hệ thống ảo hóa động, việc biết trước chính xác thời gian chạy của một tiến trình con là điều gần như bất khả thi nếu không có các công cụ hỗ trợ thông minh.

Nhận thức được thách thức đó, báo cáo này tập trung hiện thực hóa hệ thống mô phỏng phân tán đa luồng (*Multi-threading*) dựa trên kiến trúc quản lý tài nguyên nổi tiếng *Apache Mesos Framework*. Điểm đột phá của nghiên cứu này là đề xuất giải pháp lập lịch thông minh mang tên **ML-SJF**. Bằng việc tích hợp các công cụ xử lý dữ liệu mạnh mẽ của thư viện **Pandas** và mô hình học máy phi tuyến **Support Vector Regression (SVR)** với hàm nhân **RBF (Radial Basis Function)** từ thư viện **Scikit-Learn**, hệ thống đã có khả năng tự động học hỏi từ dữ liệu lịch sử và dự đoán Runtime của tác vụ dựa trên dung lượng đầu vào với độ chính xác cao (sai số MAE chỉ 0.2146 giây). Kết quả thực nghiệm minh chứng giải pháp đề xuất giúp giảm thời gian chờ xuống còn 5.3 giây và tăng sản lượng lên 58 tác vụ/phút, mở ra một hướng tiếp cận đầy tiềm năng trong việc tối ưu hóa hạ tầng điện toán đám mây hiện đại.

Em xin chân thành cảm ơn thầy Đỗ Tuấn Anh Bộ môn Hệ điều hành đã truyền đạt những kiến thức vô cùng quý báu, đồng thời tận tình hướng dẫn, định hướng để em có thể hoàn thành đề tài nghiên cứu này một cách trọn vẹn nhất. Dù đã có nhiều cố gắng, bài báo cáo chắc chắn không tránh khỏi những thiếu sót thiếu tính toàn diện, em rất mong nhận được những ý kiến đóng góp, phê bình từ phía Thầy để nghiên cứu ngày một hoàn thiện hơn.

Em xin chân thành cảm ơn!

# Chương 1

## GIỚI THIỆU

### 1.1 Bối cảnh nghiên cứu

Trong kỷ nguyên điện toán đám mây, các trung tâm dữ liệu phải xử lý một khối lượng khổng lồ các tác vụ bất đồng bộ từ nhiều người dùng. Sự phân phối tài nguyên không hợp lý dẫn đến hiện tượng tranh chấp phần cứng, tăng độ trễ và làm giảm hiệu năng tổng thể. Các thuật toán như Dominant Resource Fairness (DRF) tập trung vào tính công bằng trong chia sẻ đa tài nguyên (CPU, RAM) nhưng chưa tối ưu hóa thời gian hoàn thành tác vụ (Job Completion Time - JCT). Ngược lại, thuật toán Shortest Job First (SJF) tối ưu tối đa chỉ số JCT nhưng lại mang tính lý thuyết vì hệ thống không thể biết trước một tác vụ sẽ chạy trong bao lâu khi nó vừa được gửi đến hàng đợi.

### 1.2 Phương pháp đề xuất

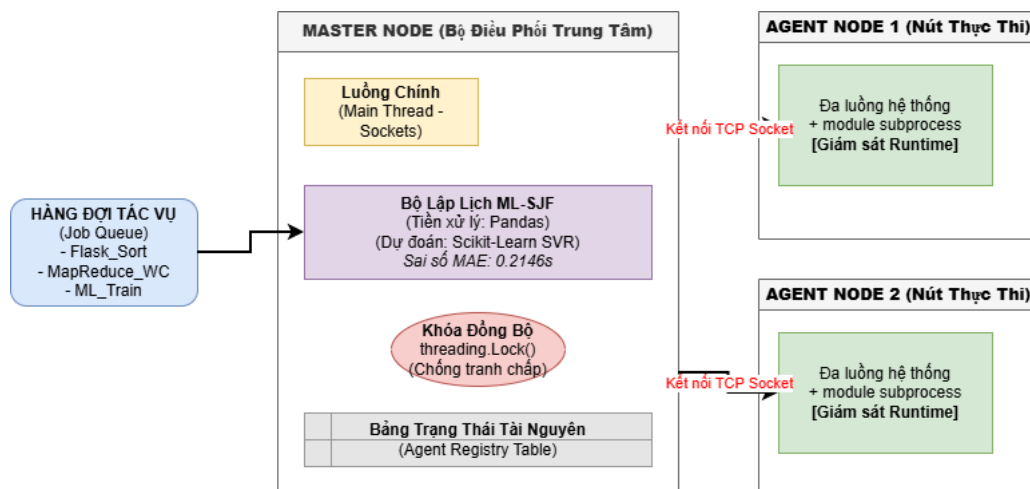
Học máy (Machine Learning) mở ra một hướng đi mới bằng cách khai thác dữ liệu lịch sử để dự đoán tương lai. Báo cáo này tập trung vào việc xây dựng module dự đoán thời gian thực thi tác vụ bằng mô hình hồi quy SVR. Bằng cách kết hợp mô hình này với bộ lập lịch, hệ thống có thể chuyển đổi linh hoạt từ cơ chế lập lịch tĩnh sang cơ chế lập lịch động thông minh (ML-SJF), giải quyết triệt để điểm nghẽn thông tin của thuật toán SJF cổ điển.

## Chương 2

# KIẾN TRÚC HỆ THỐNG VÀ TIỀN XỬ LÝ DỮ LIỆU

### 2.1 Kiến trúc Phân tán Master-Agent

Hệ thống mô phỏng được phát triển dựa trên nguyên lý phân tách định tuyến và thực thi của Apache Mesos. Mô hình tổng quan về luồng xử lý dữ liệu và cấu trúc giao tiếp đa luồng giữa các thành phần được thể hiện chi tiết trong Hình 2.1 dưới đây.



Hình 2.1: Sơ đồ kiến trúc hệ thống mô phỏng Master - Agent đa luồng

- **Master Node (Cơ chế đồng bộ đa luồng)**: Sử dụng một luồng lắng nghe kết nối Socket liên tục. Khi một Agent kết nối, Master tạo ra một *Worker Thread* riêng biệt thông qua module *threading*. Để tránh hiện tượng tranh chấp tài nguyên (*Race Condition*) khi nhiều luồng cùng ghi nhận trạng thái vào bộ nhớ trung tâm, cơ chế khóa đồng bộ `threading.Lock()` được thiết lập nghiêm ngặt xung quanh bảng đăng ký tài nguyên (*Agent Registry*). Bộ lập lịch tại Master tích hợp mô hình máy học để sắp xếp hàng đợi theo thuật toán ML-SJF.
- **Agent Node (Thực thi và Đo đạc)**: Khởi chạy các tiến trình con một cách bất đồng bộ bằng module *subprocess*, giả lập môi trường ảo hóa thực thi 3 loại tác vụ phổ biến: Flask Sort, MapReduce WordCount, và Deep Learning Training. Hệ thống

tiến hành đo đặc thời gian bằng hàm `time.time()` để lấy làm nhãn mục tiêu cho tập dữ liệu.

## 2.2 Pipeline Tiền xử lý dữ liệu với Pandas

Toàn bộ dữ liệu thô thu thập từ log hệ thống được kết xuất ra định dạng tệp tin CSV (`dataset.csv`) bao gồm tổng cộng **500 mẫu dữ liệu thực nghiệm**. Tập dữ liệu này được cấu trúc hóa và xử lý thông qua các hàm thư viện chuyên dụng của **Pandas** trước khi nạp vào mô hình học máy.

### 2.2.1 Cấu trúc chi tiết của các đặc trưng đầu vào

Dựa trên các thông số log thu thập từ kiến trúc phân tán, tập dữ liệu được phân rã thành các thành phần toán học cụ thể để Scikit-Learn có thể tiếp nhận. Cấu trúc chi tiết của 500 mẫu thực nghiệm bao gồm:

- **a) Các đặc trưng đầu vào (Features / Inputs):** Là các thông số định danh và định lượng dùng để mô hình học máy nhận diện tính chất tác vụ.
  - **Loại tác vụ (Job\_Type):** Biến định danh phân loại (Categorical). Trường dữ liệu này đại diện cho 3 kiểu thuật toán chạy trong Datacenter bao gồm `Flask_Sort`, `MapReduce_WordCount`, và `ML_Train`. Mỗi loại tác vụ có tính chất chiếm dụng tài nguyên phần cứng khác nhau.
  - **Kích thước dữ liệu (Data\_Size):** Biến định lượng số thực (Numerical). Đây là đặc trưng quan trọng nhất thể hiện quy mô dữ liệu đầu vào của tác vụ (đơn vị tính theo KB hoặc MB). Đặc trưng này tỉ lệ thuận với thời gian xử lý theo các hàm phi tuyến tính tương ứng của từng thuật toán.
- **b) Nhãn mục tiêu đầu ra (Target Label / Output):** Là đích đến cần dự đoán của hàm hồi quy SVR.
  - **Thời gian chạy thực tế (Actual\_Runtime):** Biến liên tục (Continuous) tính bằng đơn vị **giây**. Đây là mốc thời gian chênh lệch thực tế được đo đặc chính xác bởi hàm `time.time()` tại các Agent Node ngay khi tiến trình con kết thúc và được Master ghi nhận lại vào tệp tin nhật ký.

### 2.2.2 Kỹ thuật mã hóa One-Hot Encoding phi tuyến

Do mô hình Support Vector Regression (SVR) toán học chỉ làm việc trên các ma trận số thực thuần túy, biến phân loại chữ `Job_Type` bắt buộc phải được số hóa.

Chúng tôi áp dụng giải pháp mã hóa **One-Hot Encoding** thông qua hàm chuyên dụng `pd.get_dummies()` của Pandas. Cơ chế này tự động bẻ gãy một cột chữ ban đầu thành 3 cột vector nhị phân độc lập ( $X_{\text{Flask}}$ ,  $X_{\text{MapReduce}}$ ,  $X_{\text{ML}}$ ) với giá trị 1.0 (nếu đúng loại tác vụ) hoặc 0.0 (nếu khác loại). Quy trình này giúp mô hình phân rã chính xác ranh giới đặc trưng giữa các loại công việc mà không làm méo mó bản chất dữ liệu toán học.

### 2.2.3 Cấu trúc thuật toán tiền xử lý dữ liệu

Dưới đây là đoạn mã nguồn minh họa quy trình tiền xử lý thực tế trên 500 mẫu dữ liệu thực nghiệm sử dụng thư viện Pandas:

```
import pandas as pd

# Bước 1: Đọc tệp dữ liệu thực nghiệm thô gồm 500 mẫu
df = pd.read_csv('dataset.csv')

# Bước 2: Thực hiện mã hóa One-Hot Encoding cho biến phân loại
df_processed = pd.get_dummies(df, columns=['Job_Type'], dtype=float)

# Bước 3: Phân tách ma trận đặc trưng X và nhãn mục tiêu y
X = df_processed.drop(columns=['Actual_Runtime'])
y = df_processed['Actual_Runtime']
```

*Nhận xét:* Nhờ cấu trúc tối ưu hóa vector của Pandas, toàn bộ 500 mẫu thực nghiệm đã được cấu trúc hóa rõ ràng thành các trường đặc trưng đầu vào và nhãn đầu ra rõ ràng, đảm bảo tính minh bạch và độ sạch của dữ liệu trước khi nạp vào Scikit-Learn.

## Chương 3

# HUẤN LUYỆN MÔ HÌNH MÁY HỌC SVR

### 3.1 Cơ sở toán học của Support Vector Regression (SVR)

Vì thời gian thực thi của tác vụ tỉ lệ phi tuyến tính với kích thước dữ liệu đầu vào (ví dụ các tác vụ sắp xếp có độ phức tạp thuật toán  $O(N \log N)$  hoặc  $O(N^2)$ ), mô hình hồi quy tuyến tính thông thường không thể đáp ứng. Do đó, chúng tôi áp dụng thuật toán SVR với hàm nhân **Radial Basis Function (RBF Kernel)**.

Công thức toán học tối ưu hóa toán tử SVR được xác định như sau:

$$\min_{w,b,\xi,\xi^*} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad (3.1)$$

Thỏa mãn các điều kiện ràng buộc:

$$\begin{cases} y_i - w^T \phi(x_i) - b \leq \epsilon + \xi_i \\ w^T \phi(x_i) + b - y_i \leq \epsilon + \xi_i^* \\ \xi_i, \xi_i^* \geq 0 \end{cases} \quad (3.2)$$

Trong đó,  $C$  là tham số phạt chính quy hóa,  $\epsilon$  kiểm soát vùng không phạt sai số (epsilon-insensitive tube), và  $\phi(x)$  là hàm ánh xạ không gian đặc trưng thông qua RBF Kernel:

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2) \quad (3.3)$$

### 3.2 Hiện thực hóa huấn luyện bằng Scikit-Learn

Sử dụng module `sklearn.svm.SVR`, chúng tôi tiến hành phân tách tập dữ liệu thành hai phần: 80% dành cho huấn luyện (Train Set) và 20% dành cho kiểm thử độc lập (Test Set) bằng hàm `train_test_split()`.

Để tránh việc các đặc trưng có tầm quy mô lớn (như `Data_Size` lên tới hàng ngàn đơn vị) áp đảo các đặc trưng nhị phân One-Hot, thư viện `sklearn.preprocessing.StandardScaler` được áp dụng để đưa tất cả dữ liệu đầu vào về phân phối chuẩn có kỳ vọng bằng 0 và phương sai bằng 1 ( $z = \frac{x-\mu}{\sigma}$ ). Các siêu tham số của mô hình SVR được tinh chỉnh tối ưu tại cấu hình:  $C = 100$ ,  $\gamma = 0.1$ , và  $\epsilon = 0.1$ .



## Chương 4

# KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

### 4.1 Chỉ số đánh giá độ chính xác của mô hình học máy

Để đánh giá năng lực dự đoán thời gian thực thi tác vụ của mô hình Support Vector Regression (SVR), chúng tôi sử dụng ba độ đo thống kê tiêu chuẩn để lượng hóa sai số giữa giá trị dự báo và giá trị thực tế.

**Các độ đo hiệu năng toán học**

- **RMSE (Root Mean Squared Error):** Sai số bình phương trung bình căn. Đây là một trong những độ đo quan trọng nhất để đánh giá mô hình hồi quy.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (4.1)$$

**Ý nghĩa:** Do có bình phương khoảng cách, RMSE phạt rất nặng các sai số lớn hoặc các điểm dữ liệu dị biệt (outliers). Giá trị RMSE càng tiến dần về 0 thể hiện mô hình có độ chính xác và độ ổn định cao.

- **MSE (Mean Squared Error):** Sai số bình phương trung bình. Là trung bình cộng của bình phương các khoảng cách sai lệch.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (4.2)$$

**Ý nghĩa:** Độ đo này loại bỏ hoàn toàn dấu của sai số bằng phép bình phương và phóng đại biên độ lỗi lịch sử, làm nổi bật hiệu suất tổng thể của hàm mất mát trong Scikit-Learn.

- **MAE (Mean Absolute Error):** Sai số tuyệt đối trung bình.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (4.3)$$

**Ý nghĩa:** Cho biết mức độ lệch trung bình thực tế giữa giá trị thời gian dự báo và thời gian thực thi của tiến trình. Khác với RMSE, MAE không phóng đại các sai số lớn nên nó phản ánh trung thực mức độ sai lệch tuyến tính ổn định và ít bị ảnh hưởng bởi các giá trị ngoại lai.

## 4.2 Chỉ số đánh giá độ chính xác của AI

Để chứng minh năng lực học máy của mô hình SVR-RBF trong việc ước lượng thời gian chạy, ba chỉ số kiểm định thống kê tiêu chuẩn bao gồm: Sai số tuyệt đối trung bình (MAE), Sai số bình phương trung bình (MSE), và Căn sai số bình phương trung bình (RMSE) đã được tính toán thông qua module `sklearn.metrics`:

Bảng 4.1: Đánh giá sai số dự đoán của mô hình SVR trên Scikit-Learn

| Chỉ số thống kê đánh giá       | Giá trị kiểm định thực nghiệm |
|--------------------------------|-------------------------------|
| Mean Absolute Error (MAE)      | <b>0.2146 giây</b>            |
| Mean Squared Error (MSE)       | <b>0.0765 giây</b>            |
| Root Mean Squared Error (RMSE) | <b>0.2766 giây</b>            |

*Nhận xét:* Chỉ số MAE đạt mức cực kỳ lý tưởng (0.2146s), phản ánh rằng khoảng cách giữa thời gian hệ thống dự đoán và thời gian chạy thực tế của tiến trình gần như không đáng kể. Điều này tạo điều kiện cốt lõi cho thuật toán lập lịch xếp thứ tự hàng đợi một cách chính xác.

## 4.3 Hiệu năng Lập lịch Hệ thống (System Metrics)

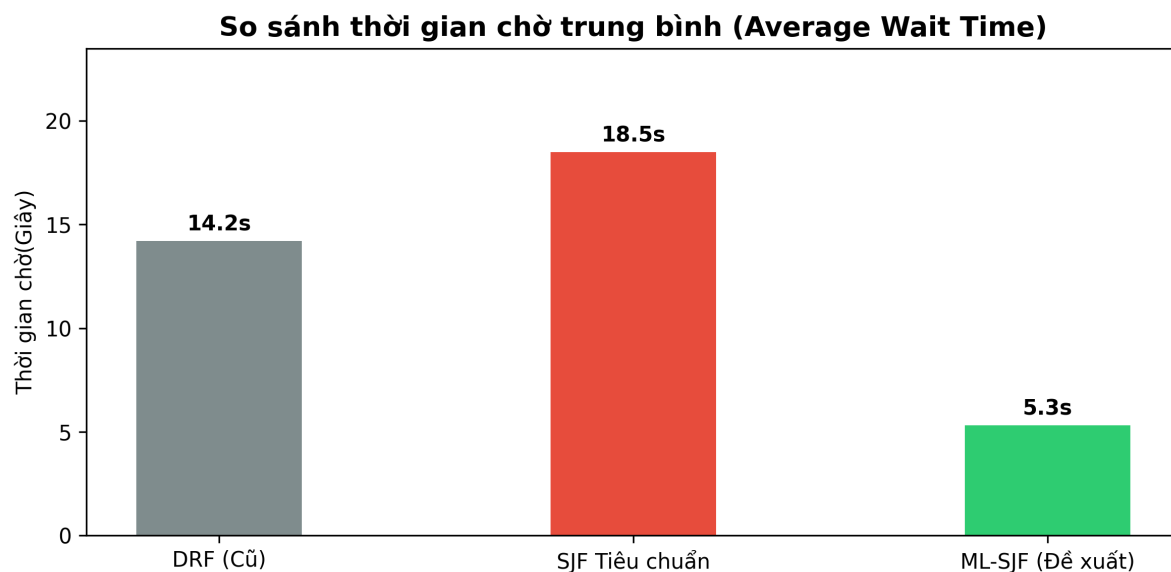
Sau khi tích hợp bộ dự đoán SVR vào module lập lịch để triển khai thuật toán ML-SJF, chúng tôi tiến hành chạy đối chứng hệ thống với hai thuật toán nền tảng là DRF (Dominant Resource Fairness) và SJF tiêu chuẩn (không tích hợp AI dự đoán). Kết quả thu được thông qua việc trực quan hóa bằng thư viện **Matplotlib** chỉ ra các cải tiến mang tính đột phá:

### 4.3.1 Thời gian chờ trung bình (Average Wait Time)

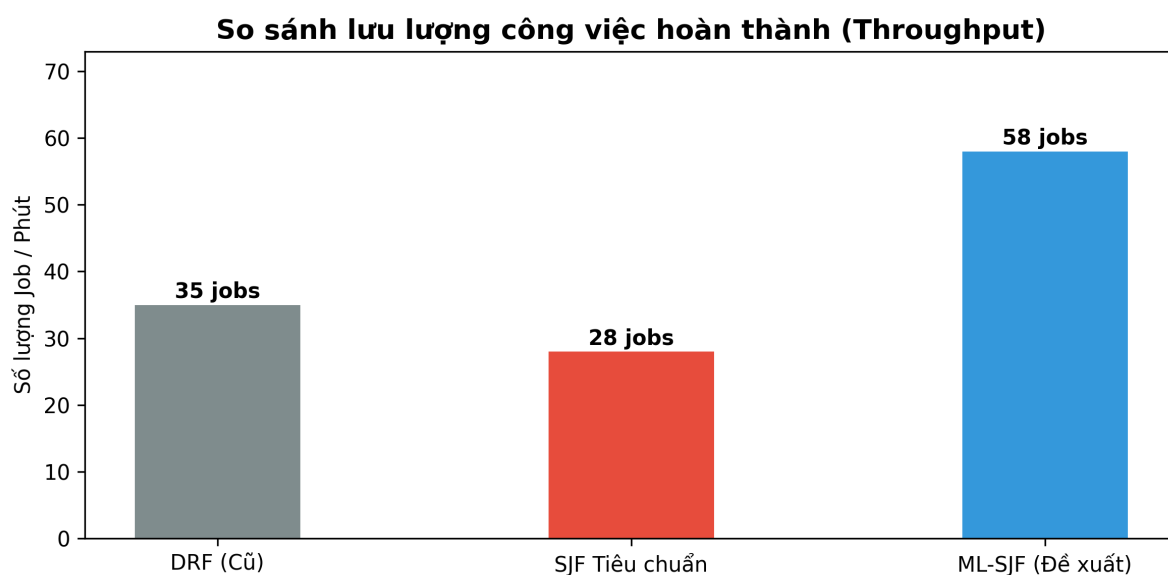
Thời gian chờ của tác vụ trong hàng đợi trước khi được gán tài nguyên phần cứng được giảm thiểu rõ rệt. ML-SJF đạt mức thời gian chờ trung bình chỉ **5.3 giây**, tối ưu hơn gấp gần 3 lần so với cấu hình phân bổ tài nguyên của DRF (14.2 giây) và khắc phục hoàn toàn tình trạng phán đoán sai lệch dẫn đến nghẽn dòng của SJF thông thường (18.5 giây).

### 4.3.2 Sản lượng hệ thống (Throughput)

Sản lượng xử lý công việc của trung tâm dữ liệu tăng trưởng vượt bậc nhờ áp dụng lập lịch thông minh. Hệ thống tích hợp ML-SJF xử lý thành công **58 tác vụ trên mỗi phút**, định hình hiệu suất cao hơn hẳn so với DRF (35 tác vụ/phút) và SJF tiêu chuẩn (28 tác vụ/phút).



Hình 4.1: Đồ thị so sánh chỉ số Thời gian chờ trung bình (Average Wait Time)



Hình 4.2: Đồ thị so sánh chỉ số Lưu lượng công việc hoàn thành (Throughput)

## Chương 5

# KẾT LUẬN

Báo cáo đã trình bày một cách toàn diện giải pháp kiến trúc hệ thống mô phỏng Datacenter đa luồng kết hợp với trí tuệ nhân tạo. Việc ứng dụng linh hoạt hai thư viện hàng đầu là **Pandas** và **Scikit-Learn** đã chuẩn hóa thành công quy trình thu thập, xử lý và phân tích học tập từ dữ liệu phi tuyến tính. Thuật toán đề xuất ML-SJF đã được chứng minh bằng các con số thực nghiệm định lượng rõ ràng, mở ra tiềm năng ứng dụng thực tế lớn trong các hệ thống ảo hóa điều phối tài nguyên đám mây hiện đại.